

Architectural and Implementation Notes — Backend Foundation Setup to Security

Author: Abdul Mazid

Repository: [backend-foundation-setup-to-security](#)

Context: Production-ready backend architecture using Express, TypeScript, and Neon PostgreSQL.

Last Updated: 2026-05-23

📁 ১. প্রজেক্ট স্ট্রাকচার (Project Structure) — Separation of Concerns (SoC)

```
BackendFoundationSetupToSecurity/
├── .env                                ← Environment variables (local state configuration)
├── package.json                       ← Project manifest, metadata, and dependencies
├── package-lock.json                 ← Deterministic dependency tree locking
├── tsconfig.json                     ← TypeScript Compiler (tsc) options
├── node_modules/                     ← Resolvable external dependencies
├── src/                               ← Source directory
│   ├── app.ts                        ← Router registration and middleware setup (Stateless
│   │                               Server)
│   ├── index.ts                      ← Server bootstrapping and process lifecycle
│   └── management
│       ├── config/
│       │   └── index.ts              ← Environment configuration schema and validation
│       └── db/
│           └── index.ts              ← Database connection pooling and Schema DDL execution
```

🔗 স্ট্রাকচারের বিস্তারিত বিশ্লেষণ (Architectural Rationale)

- **Separation of Concerns (SoC):** এটি একটি সফটওয়্যার আর্কিটেকচারাল ডিজাইন প্যাটার্ন, যা অ্যাপ্লিকেশনের বিভিন্ন কার্যকারিতাকে পৃথক মডিউলে বিভক্ত করে।
 - **app.ts** শুধুমাত্র এক্সপ্রেস অ্যাপ্লিকেশনের কনফিগারেশন, রাউটিং এবং মিদলওয়্যার রেজিস্ট্রেশন হ্যান্ডেল করে। এটি নেটওয়ার্কের কোনো পোর্টের সাথে সরাসরি যুক্ত হয় না, ফলে এটি একটি **Stateless Application Module**।
 - **index.ts** হলো অ্যাপ্লিকেশনের **Entry Point** বা বুটস্ট্রাপ ফাইল। এটি ওএস (OS) এবং নেটওয়ার্ক লেভেলের পোর্ট বাইন্ডিং, প্রসেস লাইফসাইকেল এবং ডাটাবেজ ইনিশিয়ালাইজেশন সিকোয়েন্স কন্ট্রোল করে।
- **সুবিধাসমূহ:**
 - **Maintainability:** প্রতিটি মডিউলের দায়িত্ব সুনির্দিষ্ট (High Cohesion)। ডাটাবেজের পরিবর্তনে রাউটিং কোড প্রভাবিত হয় না (Low Coupling)।
 - **Testability:** আমরা সার্ভারকে কোনো পোর্টে বাইন্ড না করেই **app.ts** মডিউলটি ইম্পোর্ট করে সুপারটেস্ট (**supertest**) বা অন্যান্য লাইব্রেরি দিয়ে ইন্টিগ্রেশন টেস্টিং চালাতে পারি।

📄 ২. ফাইল ১: **package.json** — প্রজেক্টের কনফিগারেশন ও ম্যানিফেস্ট

```
{
  "name": "backendfoundationsetuptosecurity",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "tsx watch ./src/index.ts"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "devDependencies": {
    "@types/express": "^5.0.6",
    "@types/node": "^25.9.1",
    "@types/pg": "^8.20.0",
    "tsx": "^4.22.3"
  },
  "dependencies": {
    "@neondatabase/serverless": "^1.1.0",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "pg": "^8.21.0"
  }
}
```

🔧 লাইন-বাই-লাইন বেসিক টু ডিপ ডাইভ বিশ্লেষণ:

১. "name": "backendfoundationsetuptosecurity"

- **Basic:** এটি আপনার প্যাকেজ বা অ্যাপ্লিকেশনের নাম।
- **Deep Dive:** npm (Node Package Manager) এই নামটিকে প্যাকেজ আইডেন্টিফায়ার হিসেবে ব্যবহার করে। যদি এই প্রজেক্টটি কোনো পাবলিক বা প্রাইভেট রেজিস্ট্রি (যেমন npmjs.com) তে রেজিস্টার করা হয়, তবে এই নামেই এটি ইউনিকলি চিহ্নিত হবে। নামটিতে কোনো ক্যাপিটাল লেটার বা স্পেস থাকতে পারবে না কারণ এটি URL-এর সাব-পাথ এবং ফাইল সিস্টেমের ডিরেক্টরি নেম হিসেবে সরাসরি ব্যবহৃত হয়।

২. "version": "1.0.0"

- **Basic:** এটি প্রজেক্টের বর্তমান সংস্করণ বা ভার্সন নম্বর।
- **Deep Dive:** এটি **Semantic Versioning (SemVer)** নীতি অনুসরণ করে। এর ফরম্যাট হলো **MAJOR.MINOR.PATCH**।
 - **MAJOR:** যখন এমন পরিবর্তন করা হয় যা পূর্ববর্তী সংস্করণের সাথে সামঞ্জস্যপূর্ণ নয় (Breaking Changes)।
 - **MINOR:** যখন অ্যাপ্লিকেশনে নতুন কোনো ফিচার যোগ করা হয়, তবে তা ব্যাকওয়ার্ড-কম্প্যাটিবল (Backward-compatible)।
 - **PATCH:** যখন সিস্টেমে কোনো বাগ বা ত্রুটি সংশোধন করা হয় যা কোনো এপিআই (API) পরিবর্তন করে না।

৩. "main": "index.js"

- **Basic:** এটি প্যাকেজের এন্ট্রি পয়েন্ট ডিক্লেয়ার করে।
- **Deep Dive:** এই মডিউলটি যদি অন্য কোনো প্রজেক্টে ডিপেনডেন্সি হিসেবে ইম্পোর্ট করা হয়, তবে Node.js মডিউল রেজোলিউশন অ্যালগরিদম প্রথমে এই ফাইলের এক্সপোর্ট করা ডাটা রিড করবে। আমাদের অ্যাপ্লিকেশনে আমরা সরাসরি `tsx` দিয়ে সোর্স কোড চালাচ্ছি, তাই এই কনফিগারেশনটি লোকাল রানটাইমকে প্রভাবিত করছে না।

8. "scripts": { "dev": "tsx watch ./src/index.ts" }

- **Basic:** এটি একটি শর্টকাট কমান্ড যা টার্মিনালে `npm run dev` লিখে রান করা যায়।
- **Deep Dive:**
 - `tsx` (TypeScript eXecute) হলো একটি esbuild-চালিত CLI টুল। প্রথাগত compilers (যেমন: `ts-node`) প্রতিটি পরিবর্তনে রি-টাইপচেক করে যা রানটাইমকে ধীর করে দেয়। `tsx` টাইপচেক এড়িয়ে সরাসরি TypeScript AST (Abstract Syntax Tree) কে দ্রুত জাভাস্ক্রিপ্ট কোডে রূপান্তর (Transpile) করে Node.js-এ পাঠায়।
 - `watch` ফ্ল্যাগটি ওএস-লেভেল ফাইল সিস্টেম ইভেন্টস (যেমন ফোল্ডারে কোনো ফাইল সেভ হওয়া) ট্র্যাক করে নোড প্রসেসটিকে রিস্টার্ট করে।

৫. "type": "module"

- **Basic:** এটি Node.js-কে নির্দেশ করে যে আমরা ES Modules (ESM) ব্যবহার করব।
- **Deep Dive:** জাভাস্ক্রিপ্টের দুটি প্রধান মডিউল সিস্টেম রয়েছে:
 - **CommonJS (CJS):** এটি নোড ডট জেএস-এর আদি মডিউল সিস্টেম। এটি সিনক্রোনাস রানটাইমে ফাইল লোড করে। এর সিনট্যাক্স হলো `const module = require('module')`।
 - **ECMAScript Modules (ESM):** এটি ইসিএমএ স্ট্যান্ডার্ড মডিউল সিস্টেম। এটি অ্যাসিঙ্ক্রোনাস এবং স্ট্যাটিকভাবে কম্পাইল-টাইমে ফাইল রেজোলিউশন করে। এর সিনট্যাক্স হলো `import module from 'module'`।
 - `"type": "module"` ডিক্লেয়ার করার ফলে Node.js প্রজেক্টের সব ফাইলকে ESM হিসেবে গণ্য করে, যার ফলে আমরা ফাইলগুলোতে ইম্পোর্ট ও এক্সপোর্ট সিনট্যাক্স ব্যবহার করতে পারি।

৬. "devDependencies" (Development Dependencies)

- **Basic:** এগুলো কেবল প্রজেক্ট ডেভেলপমেন্ট বা কোড লেখার সময় প্রয়োজন হয়, প্রোডাকশন সার্ভারে রান করার সময় দরকার হয় না।
- **Deep Dive:**
 - `@types/express` & `@types/node` & `@types/pg`: এগুলো হলো টাইপ ডেফিনেশন ফাইল (`.d.ts`) যা DefinitelyTyped থেকে আসে। এক্সপ্রেস, নোড ও পিজি লাইব্রেরিগুলো মূলত জাভাস্ক্রিপ্ট লেখা। টাইপস্ক্রিপ্ট কম্পাইলার যেন তাদের ইন্টারফেস, অবজেক্ট প্যারামিটার ও মেথডগুলো বুঝতে পারে এবং আমাদের অটো-কমপ্লিশন ও টাইপ সেফটি দিতে পারে, তাই এই টাইপ ফাইলগুলো ব্যবহৃত হয়।
 - `tsx`: ডেভেলপমেন্টে রান করার জন্য এটি ব্যবহৃত হয়, প্রোডাকশনে আমরা আগে TypeScript কম্পাইল করে `tsc` দিয়ে `.js` ফাইল তৈরি করি, তাই প্রোডাকশনে এর প্রয়োজন নেই।

৭. "dependencies" (Runtime Dependencies)

- **Basic:** এগুলো প্রোডাকশন এবং ডেভেলপমেন্ট উভয় রানটাইমে অ্যাপ্লিকেশন এক্সিকিউট করার জন্য আবশ্যিক।
- **Deep Dive:**
 - `@neondatabase/serverless`: এটি একটি পিজিকানেকশন অ্যাডাপ্টার যা WebSocket বা HTTP প্রোটোকল ব্যবহার করে ক্লাউড ডাটাবেজ Neon-এর সাথে যোগাযোগ স্থাপন করে। এটি ঐতিহ্যবাহী TCP

পোর্টের সংযোগের সীমাবদ্ধতা দূর করে।

- **dotenv**: প্রজেক্ট ডিরেক্টরিতে থাকা `.env` ফাইলের ডেটা রিড করে মেমরিতে `process.env` অবজেক্টে লোড করার সিস্টেমাস মডিউল।
- **express**: এক্সপ্রেস ফ্রেমওয়ার্ক যা Node.js এর কোর HTTP মডিউলের ওপর একটি শক্তিশালী অ্যাবস্ট্রাকশন লেয়ার তৈরি করে রাউটিং ও রিকোয়েস্ট লাইফসাইকেল সহজ করে।
- **pg (node-postgres)**: নন-ব্লকিং নোড ডট জেএস ক্লায়েন্ট যা PostgreSQL-এর সাথে নেটওয়ার্ক সকেট কানেকশন হ্যান্ডেল করে।



৩. ফাইল ২: `tsconfig.json` — TypeScript কম্পাইলার রুলস

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",
    "module": "esnext",
    "moduleResolution": "bundler",
    "target": "esnext",
    "types": [],
    "sourceMap": true,
    "declaration": true,
    "declarationMap": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "strict": true,
    "jsx": "react-jsx",
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "noUncheckedSideEffectImports": true,
    "moduleDetection": "force",
    "skipLibCheck": true
  }
}
```



প্রতিটি অপশনের বিশদ মেকানিক্স:

- **"rootDir": "./src"**: টাইপস্ক্রিপ্টকে নির্দেশ করে সোর্স ফাইলের রুট ডিরেক্টরি কোনটি। এটি কম্পাইল করার সময় ডিস্ট্রিবিউশন ফোল্ডারে ফাইল আর্কিটেকচার একই রাখতে সাহায্য করে।
- **"outDir": "./dist"**: কম্পাইল করা জাভাস্ক্রিপ্ট ফাইলগুলো যে ডিরেক্টরিতে জমা হবে (Distribution Folder)।
- **"module": "esnext"**: কম্পাইল করা কোডে জাভাস্ক্রিপ্টের আধুনিকতম মডিউল সিনট্যাক্স (ESM) বজায় রাখার রুল।
- **"moduleResolution": "bundler"**: আধুনিক বাউলারের (যেমন Vite, Webpack, esbuild) নিয়ম অনুযায়ী প্যাথের ফাইল এক্সটেনশন ছাড়াই কোড ইম্পোর্ট করার রেজোলিউশন পদ্ধতি।
- **"target": "esnext"**: কম্পাইলারকে নির্দেশ করে কোডটি কোন জাভাস্ক্রিপ্ট ভার্সনে ডাউন-কম্পাইল হবে। `esnext` অর্থ আধুনিকতম জাভাস্ক্রিপ্ট ফিচারে আউটপুট দেওয়া, কারণ আমাদের রানটাইম Node.js এগুলো সরাসরি সাপোর্ট করে।

- **"types": []**: গ্লোবাল স্কোপে কোনো থার্ডপার্টি টাইপ অটো-ইম্পোর্ট করা বন্ধ রাখা। এটি প্রজেক্টের টাইপ কোহেশন বাড়ায়।
- **"sourceMap": true**: জাভাস্ক্রিপ্ট ফাইলের পাশাপাশি **.js.map** ফাইল জেনারেট করে। এর ফলে রানটাইমে জাভাস্ক্রিপ্ট ফাইলে কোনো এরর হলে ক্র্যাশ লগে টাইপস্ক্রিপ্টের আসল ফাইলের লাইন নম্বরটি দেখা যায়।
- **"declaration": true & "declarationMap": true**: লাইব্রেরির জন্য **.d.ts** ফাইল ও তার সোর্স ম্যাপ তৈরি করে যাতে অন্য ডেভেলপার বা আইডিই আমাদের কোডের টাইপ ডেফিনেশন সরাসরি দেখতে পারে।
- **"noUncheckedIndexedAccess": true**: ডাটা সেফটির জন্য গুরুত্বপূর্ণ। অ্যারে বা অবজেক্টের ডাইনামিক ইন্ডেক্স অ্যাক্সেস করার সময় টাইপ সিস্টেমে অবধারিতভাবে **undefined** টাইপ ইউনিয়ন করে দেয়, যা রানটাইমে আউট-অফ-ব্যাউন্ড এরর থেকে বাঁচায়।
- **"exactOptionalPropertyTypes": true**: অপশনাল প্রপার্টিতে (যেমন **age?: number**) ভ্যালু না থাকা এবং ভ্যালু হিসেবে **undefined** থাকার মধ্যে পার্থক্য কঠোরভাবে বজায় রাখে।
- **"strict": true**: টাইপস্ক্রিপ্টের সকল কঠোরতা সক্রিয় করে, যেমন টাইপ নির্ধারণ না করলে **any** এরর দেওয়া (**noImplicitAny**) এবং **null/undefined** চেক বাধ্যতামূলক করা (**strictNullChecks**)।
- **"verbatimModuleSyntax": true**: ইম্পোর্টের সময় শুধুমাত্র টাইপ ইম্পোর্ট (**import type**) এবং রানটাইম ভ্যালু ইম্পোর্টকে আলাদা রাখতে বাধ্য করে। ফলে রানটাইমে ডেটা বান্ডলিং অপ্টিমাইজড হয়।
- **"isolatedModules": true**: প্রতিটি ফাইলকে একক মডিউল হিসেবে ট্রান্সপাইল করার নিশ্চয়তা দেয়, যা নন-টিএসসি কম্পাইলারদের (যেমন **esbuild**, **babel**) জন্য ট্রান্সপিলেশন সহজ করে।
- **"moduleDetection": "force"**: ইম্পোর্ট/এক্সপোর্ট স্টেটমেন্ট না থাকলেও প্রতিটি ফাইলকে গ্লোবাল স্ক্রিপ্ট হিসেবে না ধরে একটি লোকাল মডিউল স্কোপ হিসেবে ফোর্স করে।
- **"skipLibCheck": true**: **node_modules** এ থাকা টাইপ ফাইলগুলোর টাইপচেক স্কিপ করে কম্পাইলেশন স্পিড বৃদ্ধি করে।

8. ফাইল ৩: **.env** এবং **.env.example** — পরিবেশগত স্টেট কনফিগ

```
PORT = 5000
DATABASE_URL = postgresql://neondb_owner:npg_fVX8N2ivPrmj@ep-dawn-grass-aoh6l44w-pooler.c-2.ap-southeast-1.aws.neon.tech/neondb?
sslmode=require&channel_binding=require
```

🔗 সিকিউরিটি ও প্রটোকল বিশ্লেষণ:

- **Environment Separation**: প্রজেক্টের সিক্রেট ডাটা (যেমন ডাটাবেজ পাসওয়ার্ড) এবং অ্যাপ্লিকেশনের সোর্স কোড আলাদা রাখতে হবে।
- **PostgreSQL Connection URI parsing**: **postgresql://[ইউজারনেম]:[পাসওয়ার্ড]@[হোস্ট]:[পোর্ট]/[ডাটাবেজ_নেম]?[প্যারামিটারস]**
 - **sslmode=require**: ডাটা ট্রান্সমিশনের সময় ডাটাবেজ এবং সার্ভারের মধ্যকার সংযোগটি TLS/SSL প্রটোকলে এনক্রিপ্ট করার কন্ডিশন।
 - **channel_binding=require**: SCRAM-SHA-256 চ্যানেল বাইন্ডিং যা ম্যান-ইন-দ্য-মিডল (MITM) নেটওয়ার্ক আক্রমণ প্রতিরোধ করে।

📄 ৫. ফাইল ৪: **src/config/index.ts** — সিঙ্গেল সোর্স অব ট্রুথ (Single Source of Truth)

```
import dotenv from 'dotenv';
import { env } from 'process';

dotenv.config({ quiet: true });

const config = {
  port: env.PORT as string,
  database_url: env.DATABASE_URL as string,
};

export default config;
```

লাইন-বাই-লাইন বিশ্লেষণ:

১. `import dotenv from 'dotenv';`

- **Basic:** `dotenv` লাইব্রেরিটিকে কোডে যুক্ত করা হচ্ছে।
- **Deep Dive:** এটি `node_modules` ডিরেক্টরি থেকে ESM মেকানিজমে `dotenv` মডিউলের ডিফল্ট এক্সপোর্ট ভ্যালুটি মেমরিতে ইম্পোর্ট করছে।

২. `import { env } from 'process';`

- **Basic:** নোড রানটাইমের গ্লোবাল প্রসেস থেকে `env` অবজেক্টটি নিয়ে আসা হচ্ছে।
- **Deep Dive:** Node.js-এর কোর মডিউল `process` থেকে অবজেক্ট ডিস্ট্রিকচারিংয়ের মাধ্যমে `env` অবজেক্টটি লোকাল ভ্যারিয়েবলে বাইন্ড করা হচ্ছে। এটি সরাসরি গ্লোবাল `process.env` ব্যবহার করার চেয়ে মেমরি ও স্ট্যাটিক অ্যানালাইসিসের জন্য দক্ষ।

৩. `dotenv.config({ quiet: true });`

- **Basic:** `.env` ফাইলের ডেটা নোডের এনভায়রনমেন্ট ভ্যারিয়েবলে লোড করা হচ্ছে।
- **Deep Dive:** নোড রানটাইমে ফাইল সিস্টেম সিক্সোনার্স রিডার ব্যবহার করে প্রজেক্টের রুটে থাকা `.env` ফাইলটি পড়া হয় এবং ইকুয়াল সাইন (=) দিয়ে পার্স করে এনভায়রনমেন্ট মেমরিতে কি-ভ্যালু হিসেবে ম্যাপ করা হয়। `quiet: true` প্যারামিটারটি কনফিগারেশনে পাস করার কারণে কোনো কারণে `.env` ফাইল না পাওয়া গেলে ওএস নোড রানটাইমে কোনো ওয়ার্নিং জেনারেট করে না (কারণ ডকার বা ক্লাউড কন্টেনারের ক্রেডেনশিয়াল সরাসরি ওএস এনভায়রনমেন্টে থাকে, লোকাল ফাইলে নয়)।

৪. `port: env.PORT as string`

- **Basic:** পোর্ট ভ্যারিয়েবলটি কনফিগ অবজেক্টে সেট করা হচ্ছে।
- **Deep Dive:** ওএস এনভায়রনমেন্ট ভ্যারিয়েবলের মান টাইপস্ক্রিপ্ট ডিফল্টভাবে `string | undefined` থাকে। `as string` হলো একটি **Type Assertion**। এটি টাইপ কম্পাইলারকে ইনফর্ম করে যে রানটাইমে এই ডেটা অবশ্যই স্ট্রিং হবে। (প্রোডাকশন গ্রেড কোডে এর সাথে ভ্যালিডেশন দেওয়া উচিত)।

৬. ফাইল ৫: `src/app.ts` — HTTP রাউটার ও মিডলওয়্যার কনফিগ

```
import express, { type Application, type Request, type Response } from 'express';

const app: Application = express();

app.get('/', (req: Request, res: Response) => {
  res.send("Hello world , I'm Abdul Mazid From express");
});

export default app;
```

লাইন-বাই-লাইন বিশ্লেষণ:

১. `import express, { type Application, type Request, type Response } from 'express';`

- **Basic:** এক্সপ্রেস ফাংশন এবং টাইপস্ক্রিপ্ট টাইপগুলো ইম্পোর্ট করা হচ্ছে।
- **Deep Dive:** এটি একটি মিশ্র ইম্পোর্ট সিনট্যাক্স। `express` হলো রানটাইম ভ্যানু (ডিফল্ট এক্সপোর্ট ফাংশন), এবং `type` কীওয়ার্ড দিয়ে টাইপ-অনলি ইম্পোর্ট ব্যবহার করা হয়েছে। টাইপস্ক্রিপ্ট যখন কোড ট্রান্সপাইল করবে, তখন সে `Application`, `Request`, ও `Response` টাইপ ইম্পোর্টগুলোকে আউটপুট জাভাস্ক্রিপ্ট ফাইল থেকে পুরোপুরি বাদ দেবে, কারণ জাভাস্ক্রিপ্ট রানটাইমে ইন্টারফেস বলে কিছু নেই।

২. `const app: Application = express();`

- **Basic:** এক্সপ্রেস অ্যাপ তৈরি করা হলো।
- **Deep Dive:** `express()` হলো একটি ফ্যাক্টরি ফাংশন (Factory Function)। এটি কল করার সাথে সাথে মেমরিতে একটি এক্সপ্রেস অ্যাপ্লিকেশনের স্টেট অবজেক্ট ইনস্ট্যানশিয়েট হয়। এটি Node.js-এর কোর `http.Server` লাইফসাইকেলের ওপর একটি র‍্যাপার হিসেবে কাজ করে এবং রিকোয়েস্ট লুপ হ্যান্ডেল করার মেকানিজম রেডি করে।

৩. `app.get('/', (req: Request, res: Response) => { ... });`

- **Basic:** হোম রাউটে (/) একটি গेट রিকোয়েস্ট হ্যান্ডলার তৈরি করা হয়েছে।
- **Deep Dive:**
 - `app.get()` এক্সপ্রেসের ইন্টারনাল রাউটার টেবিলে HTTP GET মেথডের জন্য / পাথের বিপরীতে একটি ইভেন্ট লিসেনার (Callback) রেজিস্টার করে।
 - `req` (Request) অবজেক্টটি আগত HTTP রিকোয়েস্টের মেটাডাটা, কুয়েরি প্যারামিটারস, বডি এবং হেডার ম্যাপ করে।
 - `res` (Response) অবজেক্টটি ক্লায়েন্টের কাছে নেটওয়ার্ক সকেটের মাধ্যমে রেসপন্স ফেরত পাঠানোর জন্য ব্যবহৃত হয়।
 - `res.send()` মেথডটি ইন্টারনালি ডেটার টাইপ চেক করে স্বয়ংক্রিয়ভাবে `Content-Type` হেডার (স্ট্রিংয়ের ক্ষেত্রে `text/html`), বাফারের সাইজ নির্ধারণ করে `Content-Length` হেডার জেনারেট করে এবং ক্লায়েন্ট ও সার্ভারের ক্যাশিং যাচাইয়ের জন্য `ETag` (Entity Tag) হেডার তৈরি করে ডাটা স্ট্রিম ক্লোজ করে দেয়।

৭. ফাইল ৬: `src/index.ts` — বুটস্ট্রাপ ও সার্ভার লাইফসাইকেল


```
import app from './app';
import config from './config';
import { initDB } from './db';

const main = async () => {
  try {
    await initDB();
    console.log("Database initialized successfully.");
    app.listen(config.port, () => {
      console.log(`Server is running on port ${config.port}`);
    });
  } catch (error) {
    console.error("Failed to start server due to database initialization failure:", error);
    process.exit(1);
  }
};

main();
```

লাইন-বাই-লাইন বিশ্লেষণ:

১. `const main = async () => { ... }`

- **Basic:** সার্ভার স্টার্ট করার মূল অ্যাসিনক্রোনাস ফাংশন।
- **Deep Dive:** অ্যাসিনক্রোনাস ও নন-ব্লকিং কোডগুলো একটি নির্দিষ্ট সিকোয়েন্সে চালানোর জন্য এই ফাংশন প্যাটার্নটি ব্যবহৃত হয়।

২. `await initDB();`

- **Basic:** ডাটাবেজ টেবিলগুলো তৈরি হওয়া পর্যন্ত অপেক্ষা করা হচ্ছে।
- **Deep Dive:** যেহেতু ডাটাবেজ ইনিশিয়ালাইজেশন একটি নেটওয়ার্ক আই/ও (Network I/O) অপারেশন, এটি একটি `Promise` রিটার্ন করে। `await` কীওয়ার্ড ব্যবহার করার কারণে জাভাস্ক্রিপ্ট ইঞ্জিন এই প্রমিজ রিজলভ হওয়া পর্যন্ত এক্সিকিউশন থ্রেডকে ব্লক না করে পরবর্তী কোড লাইনে যাওয়া স্থগিত রাখে। এর ফলে টেবিল তৈরি নিশ্চিত হওয়ার পরেই সার্ভার পোর্টে লিসেন করা শুরু করতে পারে।

৩. `app.listen(config.port, callback)`

- **Basic:** সার্ভারটি পোর্ট চালু করছে।
- **Deep Dive:** এটি ইন্টারনালি Node.js কোর নেটওয়ার্ক মডিউলের `net.Server` ব্যবহার করে ওএস (OS)-এর টিসিপি (TCP) স্ট্যাকে একটি সকেট লিসেনার ওপেন করে। পোর্টটি সফলভাবে বাইন্ড হলে ওএস থেকে ট্রিগার আসা ইভেন্টটি এক্সপ্রেসের ইভেন্ট লুপে যায় এবং কলব্যাক ফাংশনটি রান করে।

৪. `process.exit(1);`

- **Basic:** এরর হলে অ্যাপ্লিকেশন বন্ধ হয়ে যাবে।
- **Deep Dive:** নোড প্রসেস লাইফসাইকেল কন্ট্রলের জন্য এটি ব্যবহৃত হয়। ১ আর্গুমেন্টটি ওএস-কে নির্দেশ করে যে প্রসেসটি একটি আনহ্যান্ডেল্ড এরর বা ক্র্যাশের কারণে বন্ধ হয়েছে। কন্টেইনার ম্যানেজার বা পিএম২

(PM2) এর মতো রিস্টার্ট পলিসি টুলগুলো এই নন-জিরো স্ট্যাটাস কোড দেখে বুঝতে পারে যে কন্টেইনারটি আবার রিস্টার্ট করা প্রয়োজন।

📄 ৮. ফাইল ৭: `src/db/index.ts` — ডাটাবেজ ক্লায়েন্ট ও স্কিমা DDL

```
import { neon } from '@neondatabase/serverless';
import config from '../config';

export const sql = neon(config.database_url);

export const initDB = async () => {
  await sql`
    CREATE TABLE IF NOT EXISTS users (
      id SERIAL PRIMARY KEY,
      name VARCHAR(75) NOT NULL,
      email VARCHAR(255) UNIQUE NOT NULL,
      passwordHash TEXT NOT NULL,
      age INT NOT NULL,
      role VARCHAR(25) NOT NULL DEFAULT 'user',
      createdAt TIMESTAMP NOT NULL DEFAULT NOW(),
      updatedAt TIMESTAMP NOT NULL DEFAULT NOW()
    )
  `;

  await sql`
    CREATE TABLE IF NOT EXISTS orders (
      id SERIAL PRIMARY KEY,
      customerId INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
      quantity INT NOT NULL CHECK (quantity > 0),
      food TEXT NOT NULL,
      price NUMERIC(10, 2) NOT NULL,
      createdAt TIMESTAMP NOT NULL DEFAULT NOW(),
      updatedAt TIMESTAMP NOT NULL DEFAULT NOW()
    )
  `;
};
```

🔗 স্কিমা ও কুয়েরি ডিজাইন মেকানিজম:

১. `export const sql = neon(config.database_url);`

- **Basic:** Neon ডাটাবেজ কানেকশন ইনিশিয়েট করা হচ্ছে।
- **Deep Dive:** `neon` ফাংশনটি কানেকশন স্ট্রিংটি পার্স করে একটি **SQL Tagged Template Function** মেমরিতে জেনারেট করে। এই ফাংশনটি দিয়ে পরবর্তীতে আমরা ডেটাবেজে কুয়েরি করতে পারি।

২. `id SERIAL PRIMARY KEY`

- **SERIAL:** অটো-ইনক্রিমেন্টিং ইন্টিজার টাইপ। ডাটাবেজ ব্যাকএন্ডে একটি সিকোয়েন্স অবজেক্ট তৈরি করে এবং কলামটির ডিফল্ট ভ্যালু হিসেবে `nextval()` সেট করে।

- **PRIMARY KEY**: এটি নিশ্চিত করে যে টেবিলে কোনো ডুপ্লিকেট **id** থাকবে না এবং কলামটি কখনো **NULL** হতে পারবে না। ডাটাবেজ ইঞ্জিন এই কলামের ওপর একটি **B-Tree Index** তৈরি করে, যা $O(\log N)$ টাইমে ডেটা রিট্রিভ করতে সাহায্য করে।

৩. VARCHAR vs TEXT

- **VARCHAR(75)**: Variable Character. এটি ডাইনামিক মেমোরি অ্যালোকেট করে (সর্বোচ্চ ৭৫ ক্যারেক্টার)। ফিক্সড **CHAR(75)** এর মতো মেমোরি নষ্ট করে প্যাডিং দেয় না।
- **TEXT**: পাসওয়ার্ডের হ্যাশ ভার্সন অনেক বড় হতে পারে, তাই কোনো নির্দিষ্ট সাইজ লিমিট ছাড়া ডাইনামিক স্টোরেজের জন্য **TEXT** টাইপ ব্যবহার করা হয়েছে।

৪. UNIQUE ও DEFAULT

- **UNIQUE**: ইমেইল কলামে এই ইউনিক কনস্ট্রেন্ট থাকার ফলে ডাটাবেজে একই ইমেইল দিয়ে দুটি রেকর্ড তৈরি হওয়া বন্ধ হয়। ডাটাবেজ লেভেলে এর জন্য একটি ইউনিক ইনডেক্স তৈরি হয়।
- **DEFAULT 'user'**: যদি ইনসার্ট করার সময় রোলের মান না দেওয়া হয়, তবে ডাটাবেজ স্বয়ংক্রিয়ভাবে ডিফল্ট মান হিসেবে 'user' সেট করবে।

৫. রেফারেন্সিয়াল অ্যাকশন (ON DELETE CASCADE)

- **REFERENCES users(id)**: এটি একটি **Foreign Key (ফরেন কি)**। এটি নির্দেশ করে যে **orders** টেবিলের **customerId** কলামের ভ্যালুটি অবশ্যই **users** টেবিলের একটি বিদ্যমান আইডির সাথে মিলতে হবে। একে **Referential Integrity** বলে।
- **ON DELETE CASCADE**: যদি প্যারেন্ট টেবিল (**users**) থেকে কোনো ইউজার ডিলিট হয়ে যায়, তবে চাইল্ড টেবিল (**orders**) থেকে ওই ইউজারের সব অর্ডার ডাটাবেজ স্বয়ংক্রিয়ভাবে মুছে দেবে। এর ফলে সিস্টেমে কোনো **Orphaned Rows** (এমন ডেটা যার কোনো প্যারেন্ট নেই) তৈরি হতে পারে না।

৬. ডাটাবেজ লেভেল ভ্যালিডেশন (CHECK Constraint)

- **CHECK (quantity > 0)**: এটি নিশ্চিত করে যে অর্ডারের পরিমাণ সবসময় পজিটিভ হবে। কেউ ভুল করেও ঋণাত্মক বা শূন্য পরিমাণ অর্ডার ইনপুট দিতে পারবে না। ডাটাবেজ লেভেলে বিজনেস রুল প্রটেক্ট করার এটি সর্বোত্তম উপায়।

📌 ৭. আর্থিক হিসাবের জন্য টাইপ চয়েস (NUMERIC vs FLOAT)

- টাকার হিসাবের জন্য কখনো **FLOAT** বা **DOUBLE** ব্যবহার করা যাবে না। কারণ তারা বাইনারি ফ্লোটিং-পয়েন্ট রিপ্রেজেন্টেশন ব্যবহারের কারণে ভগ্নাংশের ক্ষেত্রে সঠিক মান দিতে পারে না (যেমন: $0.1 + 0.2 = 0.30000000000000004$)।
- **NUMERIC(10, 2)**: এটি একটি Fixed-point exact numeric type। এর অর্থ হচ্ছে এই কলামে সর্বোচ্চ ১০টি ডিজিট থাকতে পারবে (দশমিকের আগের ও পরের মিলে) এবং দশমিকের পরে ফিক্সড ২টি ডিজিট (যেমন: পয়সা) স্টোর করা হবে।

🔧 ৯. ভার্সন কন্ট্রোল ওয়ার্কফ্লো (Git Best Practices)

- **Conventional Commits**: কমিট মেসেজ স্ট্রাকচারাল সিমেন্টিক ফরম্যাটে হতে হবে: **type(scope): description**
 - **feat**: নতুন ফিচার যোগ করা।

- **fix**: বাগ ফিক্স করা।
- **docs**: শুধুমাত্র ডকুমেন্টেশন বা রিডমি আপডেট।
- **chore**: প্যাকেজ বা বিন্ড কনফিগ পরিবর্তন।
- **refactor**: ফিচার বা বাগ ছাড়া শুধু কোড কোয়ালিটি ইম্প্রুভমেন্ট।
- **Feature Branching**: যেকোনো পরিবর্তনের কাজ লোকাল ফিচার ব্রাঞ্চে সম্পন্ন করতে হবে এবং তা সরাসরি মেইনে পুশ না করে পুল রিকোয়েস্ট (PR) এর মাধ্যমে মার্জ করতে হবে।